CS 2210A Data Structures and Algorithms – Assignment 1

Name: Dhruwang Kishorbhai Akbari

Student No. 251186352

Q1. We need to find constants c > 0 and n_0 >= 1 integer such that,

$$f(n) \leq c \cdot g(n), for\ n \geq n_0$$

$$\frac{1}{n^2} \leq c \cdot \frac{1}{n}, for\ n \geq\ n_0$$

Multiply 'n' to both sides of the equation

$$\frac{1}{n} \leq c\ \cdot 1, for\ n \geq\ n_0$$

We choose c = 1 and we get

$$\frac{1}{n} \leq 1, for\ n \geq\ n_0$$

We see that this inequality is valid for all values of n larger than or equal to 1. So I we choose n_0 = 2, then the inequality holds true as ½ is smaller than 1. Hence we have found c = 1 and n_0 = 2, hence we have proven that $\frac{1}{n^2}\ is\ O\left(\frac{1}{n}\right)$.

Q2. We need to find constants c > 0 and n_0 >= 1 integer such that,

$$f(n) \leq c \cdot g(n), for\ n \geq n_0$$

$$\frac{1}{g(n)} \leq c\ \cdot\ \frac{1}{f(n)}, for\ n \geq n_0$$

We cross-multiply the denominators (i.e multiply g(n) and f(n) on both the sides of the inequality). We can do this because both f(n) and g(n) are positive and non-negative.

$$\frac{1}{g(n)}\ \cdot g(n) \cdot f(n) \leq c \cdot \frac{1}{f(n)} \cdot g(n) \cdot f(n)$$

$$\therefore f(n) \leq c\ \cdot g(n), for\ n \geq n_0$$

This leads us back to the definition of the "Big-Oh". We already know by the definition of "Big-Oh" that the above inequality holds true for c > 0 and n_0 >= 1. Hence we prove that $\frac{1}{g(n)}\ is\ O(\frac{1}{f(n)})$.

Q3. We want to prove that n – 1 is not O(1).

We will assume that n – 1 is O(1) and prove by contradiction that above statement is true.

We need to find constants c > 0 and n_0 >= 1 integer such that,

$$f(n) \leq c \cdot g(n), for\ n \geq n_0$$

$$n - 1 \leq c\ \cdot 1, for\ n \geq n_0$$

We add 1 to both sides of the equation

$$n \leq c + 1, for\ n \geq\ n_0$$

The inequality $n \leq c + 1$ is valid for values that are only limited to c+1, so the values of n larger than n_0 sets the inequality false. As we see that values of n are limited to $n \leq c + 1, for\ n \geq n_0$ we can conclude that the values of constant, c, can take any value of c > 0. Therefore this inequality is false and we have proved that "n – 1 is not O(1)".

Q4. MostTimes.java

Q5. The algorithm doesn't always terminate. Although the algorithm goes through the whole array and checks for x, the case in which the algorithm doesn't terminate is when the element to be found, x, is not in the array. This is when the while loop doesn't terminate and goes into an infinite loop. The algorithm always terminates when the element, x, is in the array at some place and correctly outputs the place where the element is there. For example, if we have an Array L = {5,2,3,7,6}, n = 5, x = 8. The first iteration of the loop goes through the even positions in the array and then the odd positions. We see that 8 is clearly not in the array but when the algorithm runs the value of 'i' will keep on having odd values till 5 (elements in the array), i.e 1,3,5.

Q6. The algorithm doesn't always produce a correct output. There is one flaw in this algorithm which makes the algorithm incorrect, the flaw is when, j is assigned value 1, in the 'if' condition the algorithm doesn't actually check the last element of the array and skips that element then the while-loop loops through values of 'i' and eventually terminates and returns -1. If the element, x, is anywhere else in the array then correct output is achieved. For example, Array L = {5,1,2,7,8,9}, n = 6, x = 9. Following the algorithm as it is we get this table on each different iteration of the while-loop.

Table on next page

Iteration of while loop	i	Value of j in beginning of iteration	Value of j in the end of iteration
1	0	0	2
2	1	2	4
3	2	4	1
4	3	1	3
5	4	3	1
6	5	1	3
7	6	Exits the loop as i < n is false	Exits the loop as i < n is false

As we the values of j are in the range of 0 – 4 (inclusive) but the x is in the 5th position hence, value of j is never 5 and this algorithm excludes the last element of the array. However, if the element was anywhere else in the array we would have found it and would get a correct output. Hence the correct output is not reached and this algorithm doesn't always produce a correct output.